Handling Imbalanced Datasets

- Imbalanced data arises when one class significantly outnumbers the others, leading machine learning models to favour the majority class while performing poorly on minority classes. As a result, models may become biased and predict the dominant class more frequently, often treating minority class instances as noise. This issue can produce misleading accuracy scores, since high overall accuracy may simply reflect strong performance on the majority class. In critical domains such as fraud detection and medical diagnosis, this bias is especially problematic, as it causes skewed decision boundaries, weak generalisation, and unreliable predictions for the minority class.

• 1. Use the right evaluation metrics

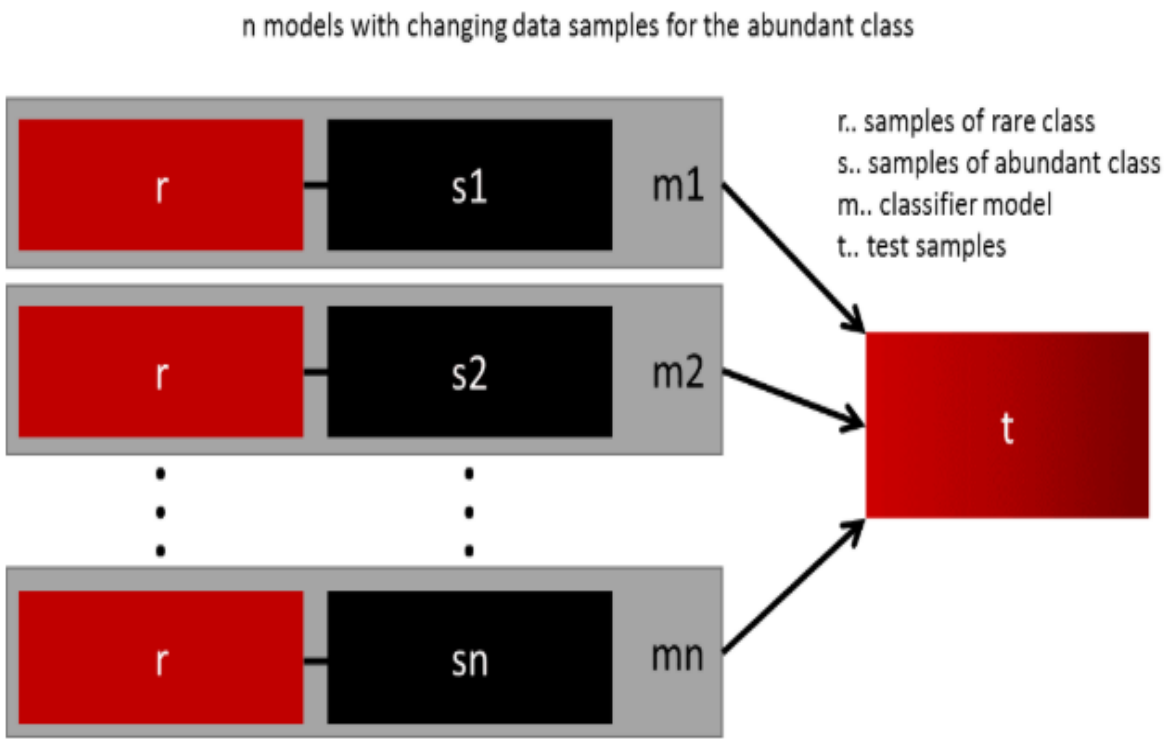
- Applying inappropriate evaluation metrics for model generated using imbalanced data can be dangerous. Imagine our training data is the one illustrated in graph above. If accuracy is used to measure the goodness of a model, a model which classifies all testing samples into “0” will have an excellent accuracy (99.8%), but obviously, this model won’t provide any valuable information for us.
- In this case, other alternative evaluation metrics can be applied such as:
- Precision/Specificity: how many selected instances are relevant.
- Recall/Sensitivity: how many relevant instances are selected.
- F1 score: harmonic mean of precision and recall.
- MCC: correlation coefficient between the observed and predicted binary classifications.
- AUC: relation between true-positive rate and false positive rate.

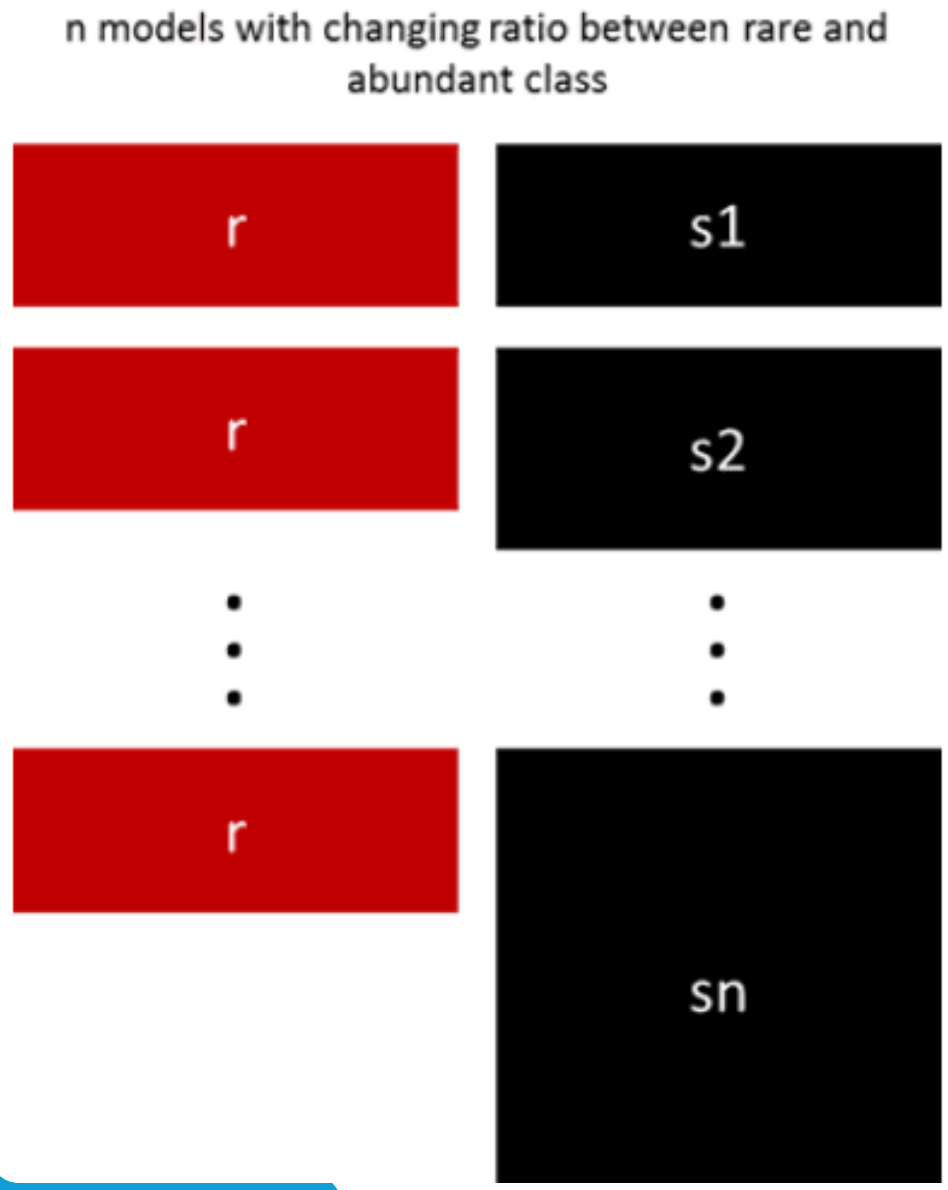
2. Resample the training set

- Apart from using different evaluation criteria, one can also work on getting different dataset. Two approaches to make a balanced dataset out of an imbalanced one are under-sampling and over-sampling.
- **Under-sampling**
- Under-sampling balances the dataset by reducing the size of the abundant class. This method is used when quantity of data is sufficient. By keeping all samples in the rare class and randomly selecting an equal number of samples in the abundant class, a balanced new dataset can be retrieved for further modelling.
- **Over-sampling**
- On the contrary, oversampling is used when the quantity of data is insufficient. It tries to balance dataset by increasing the size of rare samples. Rather than getting rid of abundant samples, new rare samples are generated by using e.g. repetition, bootstrapping or SMOTE (Synthetic Minority Over-Sampling Technique) [1].
- Note that there is no absolute advantage of one resampling method over another. Application of these two methods depends on the use case it applies to and the dataset itself. A combination of over- and under-sampling is often successful as well.

Ensemble Different Resampled Datasets

- The easiest way to successfully generalize a model is by using more data. The problem is that out-of-the-box classifiers like logistic regression or random forest tend to generalize by discarding the rare class. One easy best practice is building n models that use all the samples of the rare class and n-differing samples of the abundant class. Given that you want to ensemble 10 models, you would keep e.g. the 1.000 cases of the rare class and randomly sample 10.000 cases of the abundant class. Then you just split the 10.000 cases in 10 chunks and train 10 different models.
- This approach is simple and perfectly horizontally scalable if you have a lot of data, since you can just train and run your models on different cluster nodes. Ensemble models also tend to generalize better, which makes this approach easy to handle.

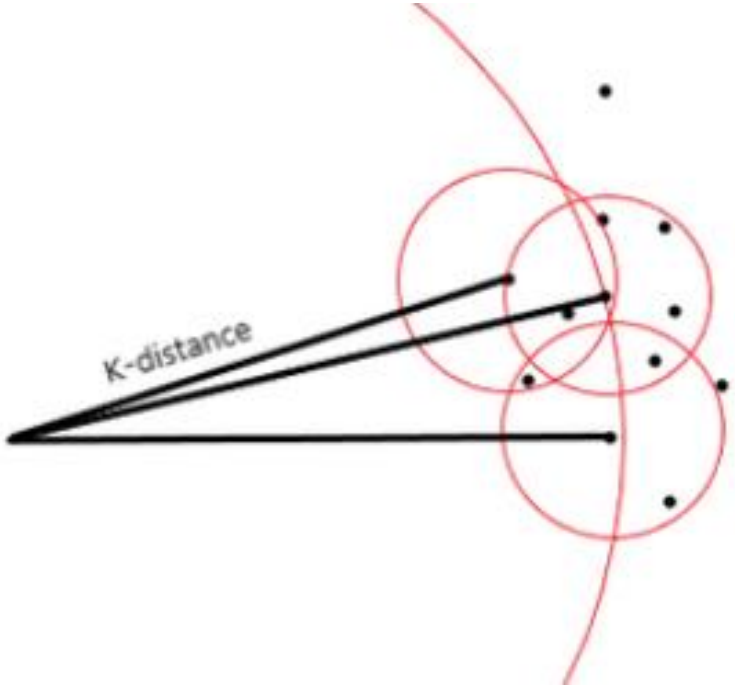




Resample with Different Ratios

- The previous approach can be fine-tuned by playing with the ratio between the rare and the abundant class. The best ratio heavily depends on the data and the models that are used. But instead of training all models with the same ratio in the ensemble, it is worth trying to ensemble different ratios. So if 10 models are trained, it might make sense to have a model that has a ratio of 1:1 (rare:abundant) and another one with 1:3, or even 2:1. Depending on the model used this can influence the weight that one class gets.

Local Outlier Factor (LOF)



Local Outlier Factor (LOF) is an unsupervised algorithm used for outlier detection. It assigns an anomaly score to each data point based on how isolated the point is relative to its surrounding neighborhood. The core idea behind LOF is to measure the deviation in local density of a data point compared to the densities of its nearest neighbors.

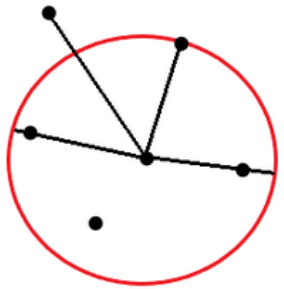
- Working of LOF
- LOF estimates the local density of a data point by analyzing the distances between the point and its *k*-nearest neighbors. By comparing the local density of a point with the densities of its neighbors, the algorithm identifies points that reside in regions of significantly lower density. Such points, which are much less dense than their surrounding neighbors, are considered outliers.
- 1. k-Distance and k-Nearest Neighbors
- The *k*-distance of a data point is defined as the distance between that point and its *k*-th nearest neighbor. This distance helps determine the local neighborhood of the point by identifying the closest *k* data points around it. The following image illustrates the concept of *k*-distance for different neighbors within a cluster surrounding a given point.

Reachability Distance

- The reachability distance is derived using the *k*-distance and is defined as the maximum of two values: the actual distance between two data points and the *k*-distance of the neighboring point. This concept helps smooth out irregularities caused by very close data points.
- Mathematically, the reachability distance between points *A* and *B* is expressed as:
- Here, *B* represents the reference (central) point, and *A* is a neighboring point.

$$\text{reachability-distance}_k(A, B) = \max\{k\text{-distance}(B),\ d(A, B)\}$$

- As illustrated in the figure above, for points located within the cluster, the *k*-distance is used, whereas for points lying outside the cluster, the actual distance between the points is considered. The reachability distances to all *k*-nearest neighbors of a data point are then computed. These values are used to calculate the Local Reachability Density (LRD) of that point, which represents how densely the point is surrounded by its neighbors.



LRD

- Local Reachability Density (LRD) quantifies the density of the *k*-nearest neighbors surrounding a given data point. It is computed as the inverse of the average reachability distance from the point to its *k*-nearest neighbors. A higher LRD value indicates that the point lies in a denser region, while a lower LRD suggests that the point is located in a sparser area.

$$\mathrm{lrd}_k(A) := \frac{1}{\left(\frac{\sum_{B \in N_k(A)} \text{reachability-distance}_k(A,B)}{|N_k(A)|}\right)}$$

- Local Outlier Factor (LOF)
- The Local Outlier Factor (LOF) of a data point is calculated as the ratio between the average Local Reachability Density (LRD) of its *k*-nearest neighbors and the LRD of the point itself. This ratio reflects how the local density of a point compares to the densities of its surrounding neighbors.
- A LOF value close to 1 indicates that the point has a density similar to its neighbors, while values significantly greater than 1 suggest that the point lies in a region of lower density and is therefore considered an outlier. The LOF is mathematically defined as follows:

$$\mathrm{LOF}_k(A) = \frac{\sum_{B \in N_k(A)} \mathrm{lrd}_k(B)}{|N_k(A)| \cdot \mathrm{lrd}_k(A)}$$

Feature Importance and Selection

- Feature selection is the process of identifying and selecting the most relevant input features for a machine learning model. By retaining only the most informative features, it enhances model performance, reduces noise, and improves the interpretability of results.
- Key benefits of feature selection include:
- Eliminating irrelevant and redundant features
- Improving model accuracy while reducing overfitting
- Decreasing training time and computational cost
- Simplifying models, making them easier to understand and interpret

Need for Feature Selection

Feature selection is a crucial step in data science and machine learning due to several important advantages:

- **Improved Accuracy:** Models perform better when trained using only the most relevant features.
- **Faster Training:** Reducing the number of features lowers computational cost and speeds up model training.
- **Greater Interpretability:** Fewer input variables make the model's behavior easier to understand and explain.